

**Tugas Kecil 1 IF2211 Strategi Algoritma**

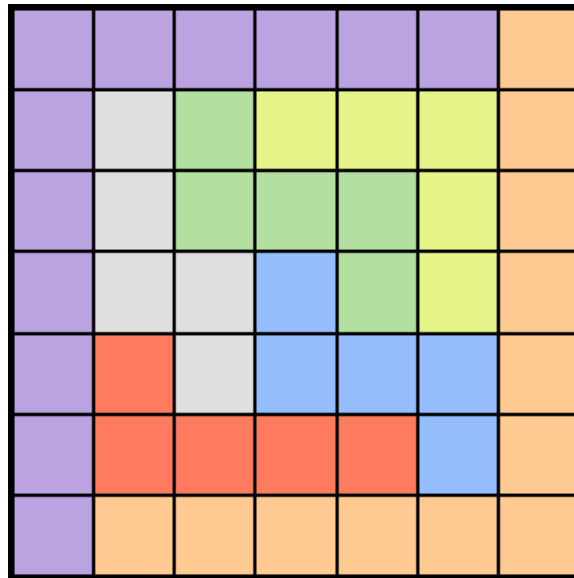
**Penyelesaian Permainan Queens**

**dengan Algoritma Brute Force**

Rasyad Satyatma - 13524142

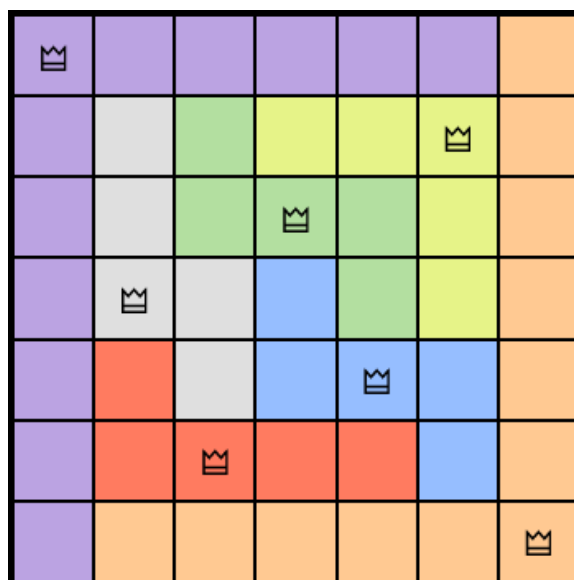
# I. Permainan Queens

Permainan Queens merupakan suatu permainan logika dimana pemain perlu meletakkan *queen* pada sebuah papan dengan aturan-aturan tentu. Pemain dapat dibilang menang ketika penempatan *queens* memenuhi aturan-aturan yang ada.



Contoh Papan Permainan Queens, <https://www.archivedqueens.com>

Sementara, konfigurasi papan yang “menang” merupakan konfigurasi papan dimana semua *queen* ditempatkan di kotak yang memenuhi aturan menang



Konfigurasi Menang Permainan Queens, <https://www.archivedqueens.com>

Diperhatikan, konfigurasi menang papan memenuhi beberapa kondisi:

1. Hanya terdapat satu *queen* tiap baris atau kolom
2. Tidak ada *queen* yang ditempatkan berdekatan dengan *queen* yang lain
3. Tiap warna/*region* memiliki hanya satu *queen*

Memahami aturan-aturan ini merupakan kunci untuk membuat penyelesaian dari permainan Queens.

## II. Batasan Input dan Output

Program menerima input melalui masukan pengguna atau file berekstensi .txt yang memiliki format tertentu.

```
AAACC  
BBBBB  
CCCCC  
DDFFF  
DDDDF
```

Input program selalu berupa daftar karakter dengan format  $N \times N$  (berbentuk kotak), dengan tiap karakter unik sebagai satu *region*. Input yang diberikan harus memiliki jumlah *region* yang sama dengan jumlah baris atau kolom. *Region* juga harus selalu kontinu dan tidak pecah, tidak boleh ada karakter yang sama menjadi dua pulau berbeda pada input.

Output program juga sebagai berikut:

```
#AACC  
BBB#C  
C#CCC  
DDFF#  
DD#FF
```

Dengan karakter-karakter input sebagai *region* awal, dan karakter # sebagai *queen*.

### III. Algoritma

Penyelesaian permainan Queens menggunakan rekursi fungsi *FindSolutions* yang menerima beberapa parameter, yaitu objek *board*, semua *region* dari papan input, index dari *region* yang ditelusuri, dan *array* dari posisi kotak yang dicoba sebagai solusi. Program melakukan iterasi untuk setiap kombinasi posisi *queen* tiap *region* secara naif, dimana program melakukan iterasi untuk suatu *region*, bahkan ketika kombinasi *queen* pada *region* sebelumnya dipastikan tidak memiliki solusi.

Cara kerja algoritma adalah dengan memilih kotak pertama pada *array* kotak yang terbagi per-*region*. Lalu melakukan iterasi tiap kombinasi pada *region* pada posisi terakhir di *array*. Jika tidak ditemukan solusi untuk tiap posisi yang mungkin, maka algoritma akan mengubah posisi *queen* pada *region* di depannya, lalu melakukan hal yang sama untuk setiap *region*, dari belakang ke depan. Proses ini dilakukan dengan melakukan rekursi untuk tiap posisi *queen* tiap *region*, sehingga tiap konfigurasi *queen* pasti akan dicari. Ketika *array* solusi sudah memiliki posisi *queen* untuk tiap *region*, akan dilakukan cek untuk konfigurasi *queen* pada posisi-posisi tersebut, jika belum valid, maka proses akan dilanjutkan dengan konfigurasi lainnya. Pada kondisi dimana posisi pada.

Proses tutur balik ini merupakan bentuk *exhaustive search*, dengan modifikasi untuk mengurangi jumlah iterasi yang diperlukan dengan menambahkan *constraint* hanya satu *queen* yang bisa ditempatkan untuk tiap *region*. Meskipun algoritma mungkin bukan *brute-force* "murni" tetapi masih dapat diklasifikasikan sebagai algoritma *brute-force*, karena memiliki karakteristik algoritma yang mencari solusi, pada posisi yang tidak mungkin, tanpa adanya pengurangan ruang solusi yang dinamis.

Jika menemukan solusi, maka algoritma akan mengembalikan nilai *true*, serta *array* solusi yang sudah terisi dengan kotak-kotak posisi queen. Apabila tidak ada solusi yang ditemukan, maka algoritma hanya mengembalikan nilai *false*.

<sup>1</sup>What even is *brute-force* murni bro

Pseudo-code program antara lain sebagai berikut:

```
func FindSolutions(b *Board, regs [][]*Cell, idx int, sols []*Cell) → bool
  if idx == len(regs) then
    for region in regs
      for cell in region
        cell.HasQueen = false # Menghapus semua queen untuk tiap region, mulai ulang

    for cell in sols
      cell.HasQueen = true # Mengembalikan queen pada yang sudah ada pada array solusi

    if ValidateBoard(b) == true then
      → true
    → false

  for cell in regs[idx]
    sols[idx] = cell # Mencoba posisi cell sekarang sebagai solusi
    if FindSolutions(b, regs, idx+1, sols) then
      → true

  → false
```

Alur algoritma secara spesifik adalah sebagai berikut:

1. Membandingkan nilai index yang diberikan dengan jumlah *region*, jika sama, maka algoritma sudah sampai pada *region* terakhir, sehingga menghapus semua *queen* pada papan lalu menambahkannya kembali untuk tiap *cell* pada *array* sols dan mengecek apakah konfigurasi papan valid, jika tidak, maka algoritma lanjut mengecek konfigurasi lainnya.
2. Jika nilai index tidak sama, maka akan melanjutkan pencarian pada *region* selanjutnya, untuk tiap posisi *queen* pada region sekarang, hingga menemukan konfigurasi papan yang valid, jika tidak ada, maka mengembalikan nilai false.

## IV. Kode Program

Keseluruhan kode program dapat ditemukan pada *repository*:

[https://github.com/PikaProgram/Tucil1\\_13524142](https://github.com/PikaProgram/Tucil1_13524142)

Board.go

```
type Cell struct {
    Row      int
    Col      int
    HasQueen bool
    RegionID string
}

type Board struct {
    Rows      int
    Cols      int
    Cells      [][]*Cell
    Adjacency  [][][]*Cell
    Regions    map[string][]*Cell
    RegionCount int
}

func (b *Board) PlaceQueen(row, col int) {
    b.Cells[row][col].HasQueen = true
}

func (b *Board) RemoveQueen(row, col int) {
    b.Cells[row][col].HasQueen = false
}

func (b *Board) hasQueen(row, col int) bool {
    return b.Cells[row][col].HasQueen
}

func (b *Board) String() string {
    var sb strings.Builder
    for r := 0; r < b.Rows; r++ {
        for c := 0; c < b.Cols; c++ {
            if b.Cells[r][c].HasQueen {
                sb.WriteString("#")
            } else {
                sb.WriteString(b.Cells[r][c].RegionID)
            }
        }
        sb.WriteString("\n")
    }
    return sb.String()
}
```

```

func NewBoard(data [][]string) (*Board, error) {
    rows := len(data)
    cols := len(data[0])

    if rows == 0 || cols == 0 {
        return nil, errors.New("Board data cannot be empty")
    }

    if rows != cols {
        return nil, errors.New("Board must be square (rows must equal columns)")
    }

    cells := make([][]*Cell, rows)
    adjacency := make([][][]*Cell, rows)

    for r := 0; r < rows; r++ {
        cells[r] = make([]*Cell, cols)
        for c := 0; c < cols; c++ {
            cell := &Cell{Row: r, Col: c, HasQueen: false, RegionID: data[r][c]}
            cells[r][c] = cell
        }
    }

    regions := make(map[string][]*Cell)
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            regionID := cells[r][c].RegionID
            regions[regionID] = append(regions[regionID], cells[r][c])
        }
    }

    if len(regions) != rows {
        return nil, errors.New("Number of regions must equal the number of
rows/columns")
    }

    for r := 0; r < rows; r++ {
        adjacency[r] = make([][]*Cell, cols)
        for c := 0; c < cols; c++ {
            var adjCells []*Cell
            for dr := -1; dr ≤ 1; dr++ {
                for dc := -1; dc ≤ 1; dc++ {
                    if dr == 0 && dc == 0 {
                        continue
                    }
                    nr, nc := r+dr, c+dc
                    if nr ≥ 0 && nr < rows && nc ≥ 0 && nc < cols {
                        adjCells = append(adjCells, cells[nr][nc])
                    }
                }
            }
            adjacency[r][c] = adjCells
        }
    }
}

```



```

    }

    return &Board{
        Rows:      rows,
        Cols:      cols,
        Adjacency:  adjacency,
        Cells:     cells,
        Regions:   regions,
        RegionCount: len(regions),
    }, nil
}

```

Solve.go

```

var iterationCount int
var elapsedTime time.Time
var lastTime time.Time

var solveStatsKey sync.RWMutex

func GetCurrentSolveStats() (iteration int, elapsed time.Duration) {
    solveStatsKey.RLock()
    defer solveStatsKey.RUnlock()
    return iterationCount, time.Since(elapsedTime)
}

func ResetSolveStats() {
    solveStatsKey.Lock()
    iterationCount = 0
    elapsedTime = time.Now()
    lastTime = time.Now()
    solveStatsKey.Unlock()
}

func addIteration() {
    solveStatsKey.Lock()
    iterationCount++
    solveStatsKey.Unlock()
}

func CreateSolvedBoard(b *Board) (*Board, error) {
    regionIDs := make([]string, 0, len(b.Regions))
    regions := make([][]*Cell, 0, len(b.Regions))
    ResetSolveStats()

    for id, cells := range b.Regions {
        regionIDs = append(regionIDs, id)
        regions = append(regions, cells)
    }
}

```

```

    sort.Strings(regionIDs)

    sols := make([]*Cell, len(regions))

    if FindSolutions(b, regions, 0, sols) {
        return b, nil
    }

    return nil, errors.New("No Solution Found")
}

func FindSolutions(b *Board, regs [][]*Cell, idx int, sols []*Cell) bool {
    if idx == len(regs) {
        for _, region := range regs {
            for _, cell := range region {
                cell.HasQueen = false
            }
        }

        for _, cell := range sols {
            cell.HasQueen = true
        }

        addIteration()

        if ValidateBoard(b) == nil {
            return true
        }
        return false
    }

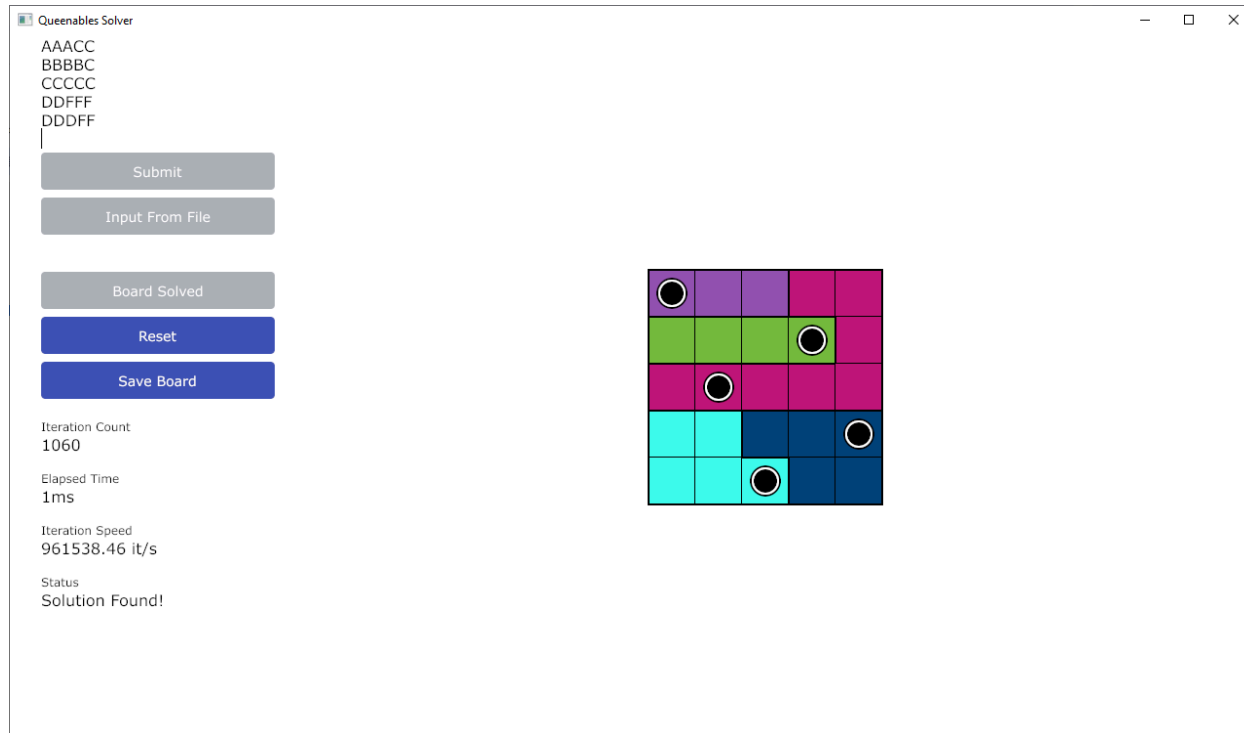
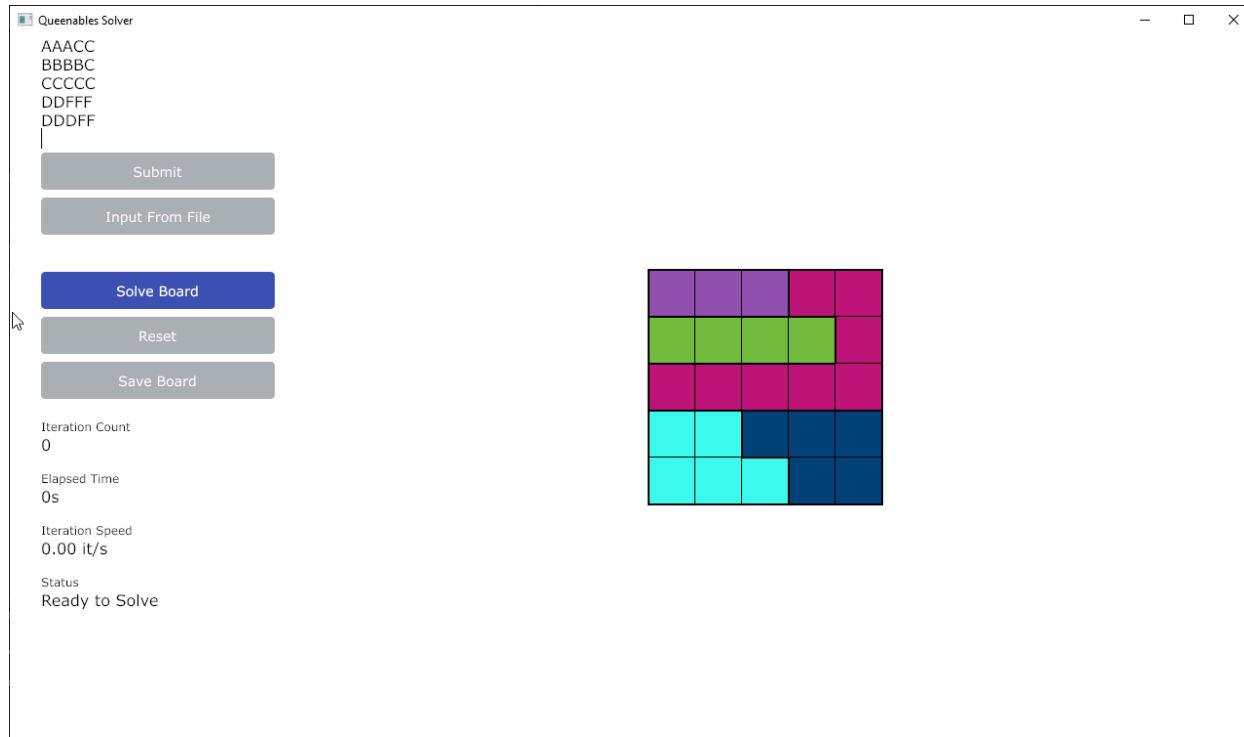
    for _, cell := range regs[idx] {
        sols[idx] = cell
        if FindSolutions(b, regs, idx+1, sols) {
            return true
        }
    }

    return false
}

```

# V. Testing

## A. Board Valid, Memiliki Solusi



## B. Board Valid, Tidak Memiliki Solusi

AA  
BB

Submit

Input From File

Solve Board

Reset

Save Board

Iteration Count  
0

Elapsed Time  
0s

Iteration Speed  
0.00 it/s

Status  
Ready to Solve



AA  
BB

Submit

Input From File

Solve Board

Reset

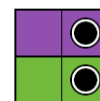
Save Board

Iteration Count  
4

Elapsed Time  
0s

Iteration Speed  
0.00 it/s

Status  
No Solution Found



C. Board Tidak Valid

WHAT  
ISST  
MAAA

Submit

Input From File

Solve Board

Reset

Save Board

D. Input Board Dari File

|AAAAAAAAA  
AABBBBBAA  
AABCCCBAA  
AABCCBBDD  
AEEBBBFFD  
EEGGBFFDD

Submit

Input From File

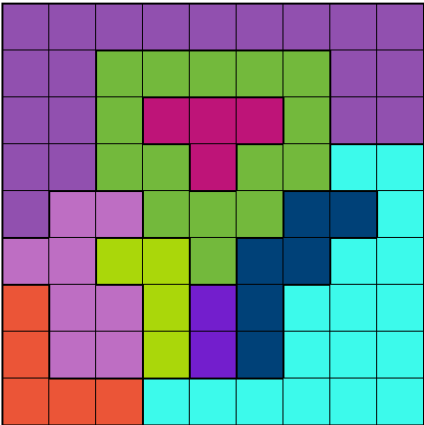
E:  
\\Dev\\Projects\\Personal\\School\\As  
signments\\Tucil1\_13524142\\test\\  
input\\2.txt

Solve Board

Reset

Save Board

Iteration Count  
0  
  
Elapsed Time  
0s  
  
Iteration Speed  
0.00 it/s  
  
Status  
Ready to Solve



E. Save To File

```

AAAAAAAAA
AABBBBBAA
AABCCCBAA
AABBCBBDD
AEEBBBFFD
EEGGBFFDD

```

Submit

Input From File

E:  
 \Dev\Projects\Personal\School\As  
 signments\Tucil1\_13524142\test\  
 input\2.txt

Board Solved

Reset

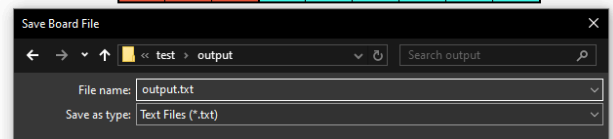
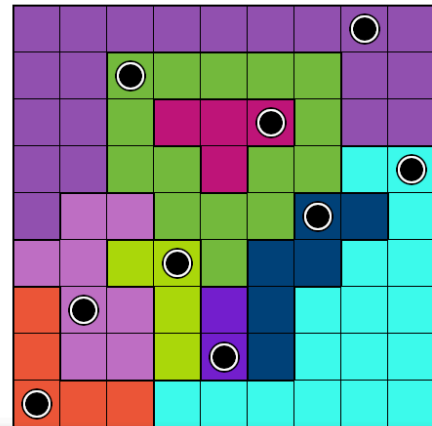
Save Board

Iteration Count  
 13783082

Elapsed Time  
 24.783s

Iteration Speed  
 556145.40 it/s

Status  
 Solution Found!



## F. File Input Dan Output

| input.txt                                                                                                         | output.txt                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| AAAAAAAAA<br>AABBBBBAA<br>AABCCCBAA<br>AABBCBBDD<br>AEEBBBFFD<br>EEGGBFFDD<br>HEEGIFDDD<br>HEEGIFDDD<br>HHHDDDDDD | AAAAAAA#A<br>AA#BBBBBAA<br>AABCC#BAA<br>AABBCBBD#<br>AEEBBB#FD<br>EEG#BFFDD<br>H#EGIFDDD<br>HEEG#FDDD<br>#HHDDDDDD |

## VI. Appendix

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Rasyad Satyatma

| No | Poin                                                                               | Ya | Tidak |
|----|------------------------------------------------------------------------------------|----|-------|
| 1  | Program berhasil di kompilasi tanpa kesalahan                                      | ✓  |       |
| 2  | Program berhasil di jalankan                                                       | ✓  |       |
| 3  | Solusi yang diberikan program benar dan mematuhi aturan permainan                  | ✓  |       |
| 4  | Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt | ✓  |       |
| 5  | Program memiliki Graphical User Interface (GUI)                                    | ✓  |       |
| 6  | Program dapat menyimpan solusi dalam bentuk file gambar                            |    | ✗     |

